

KnowledgeHub — Line-by-Line Code Walkthrough

Every file, every import, explained simply but technically System Design • Semester 4

Companion to `EXPLANATION_GUIDE.md`. That guide explains *what* the system is and *why*. **This document explains the actual code** — for each file: what every import is, **where it comes from** (Python standard library / an installed third-party library / our own project), and what each block of code does.

How to read this guide

Three kinds of imports

Every `import` line pulls a name from one of three places. Throughout this guide each import is tagged:

Tag	Meaning	Example
 stdlib	Built into Python — nothing to install	<code>import json , from datetime import datetime</code>
 third-party	An external library we installed via <code>pip</code> (listed in <code>requirements.txt</code>)	<code>from fastapi import FastAPI</code>
 internal	Our own code, from another file in the <code>app/</code> package	<code>from app.config import settings</code>

A quick rule for reading any import: - `from X import Y` → "get the name `Y` out of module `X`." - If `X` starts with `app.` → it's **our** code (internal). Otherwise it's stdlib or a library.

Recurring concepts (explained once here, used everywhere)

These appear in many files; understand them once:

- **Type hints** (`name: str` , `-> bool`): annotations saying what type a value is. Python doesn't enforce them at runtime, but libraries like Pydantic/FastAPI/SQLAlchemy *read* them to do validation and wiring. They also help your editor.
- **Decorator** (`@something` above a function): wraps the function to give it extra behaviour. `@app.get("/health")` means "run this function when someone GETs /health."
- **Depends(...)** (FastAPI): "before running my endpoint, call this other function and inject its result as an argument." This is **dependency injection** — used for getting the DB session and the current user.
- **Session** (SQLAlchemy): an open conversation with the database; you run queries through it and `commit()` to save.
- **Generator / yield** : a function that hands back a value and pauses, resuming later — used so the DB session can be opened, used, then closed.

File 1 — `app/config.py` (settings)

Purpose: load all configuration (database URL, secret key, service URLs) from environment variables / `.env`, with safe defaults.

Imports

Import	Source	What it is
<code>from pydantic_settings import BaseSettings, SettingsConfigDict</code>	 third-party (<code>pydantic-settings</code>)	<code>BaseSettings</code> = a base class that auto-reads env vars into typed fields. <code>SettingsConfigDict</code> = a callable to configure that loader.

Line by line

```
class Settings(BaseSettings):
```

Define a class that **inherits** from `BaseSettings` — so it gains the "read my fields from the environment" behaviour.

```
model_config = SettingsConfigDict(env_file=".env", extra="ignore")
```

Configure the loader: also read a `.env` file (`env_file=".env"`); ignore any extra env vars we didn't declare (`extra="ignore"`) instead of erroring.

```
app_name: str = "KnowledgeHub"
secret_key: str = "change-me"
algorithm: str = "HS256"
access_token_expire_minutes: int = 60
database_url: str = "sqlite:///./knowledgehub.db"
elasticsearch_url: str = "http://localhost:9200"
elasticsearch_index: str = "documents"
redis_url: str = "redis://localhost:6379/0"
cache_ttl_seconds: int = 300
```

Each line is `name: type = default`. The **type** is validated/converted by Pydantic; the **default** is used when the matching uppercase env var (`DATABASE_URL`, `SECRET_KEY`, ...) is absent. This is why the app runs with zero setup (SQLite + localhost) but switches to Postgres/ES in Docker just by setting env vars.

```
settings = Settings()
```

Create **one instance** — this is the moment env + `.env` are read, defaults applied, and types validated. Everything else does `from app.config import settings` and reads `settings.database_url`, etc. Effectively a single shared **config object** (singleton).

File 2 — `app/database.py` (database connection)

Purpose: create the SQLAlchemy engine (the actual DB connection), a session factory, and the `Base` class that all table models inherit from.

Imports

Import	Source	What it is
<code>from sqlalchemy import create_engine</code>	 third-party (<code>sqlalchemy</code>)	Function that builds the engine — the low-level connection pool to the database.
<code>from sqlalchemy.orm import declarative_base, sessionmaker</code>	 third-party	<code>declarative_base</code> builds the base class for ORM models; <code>sessionmaker</code> builds a factory that produces DB sessions .
<code>from app.config import settings</code>	 internal (<code>app/config.py</code>)	Our settings object — we read <code>settings.database_url</code> from it.

Line by line

```
connect_args = (  
    {"check_same_thread": False}  
    if settings.database_url.startswith("sqlite")  
    else {}  
)
```

SQLite (the zero-infra fallback) normally forbids using one connection across threads; FastAPI uses threads, so for SQLite we pass `check_same_thread=False`. For Postgres we pass nothing (`{}`). This is a one-line `if/else` expression.

```
engine = create_engine(settings.database_url, connect_args=connect_args, pool_pre_ping=True)
```

Build the **engine** from the URL. `pool_pre_ping=True` checks a pooled connection is still alive before using it (avoids "stale connection" errors).

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Create a **session factory** bound to that engine. Calling `SessionLocal()` later gives a new session. `autocommit/autoflush=False` = we control exactly when data is written (explicit `commit()`).

```
Base = declarative_base()
```

Create the **base class**. Every table model in `models.py` inherits from `Base`, which is how SQLAlchemy knows about all tables.

```
def get_db():  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()
```

A **FastAPI dependency**. It opens a session, `yield` s it to the endpoint (the endpoint runs at the `yield`), then guarantees `db.close()` afterwards (the `finally` runs even if the endpoint errors). Endpoints get a session by writing `db: Session = Depends(get_db)`.

File 3 — `app/models.py` (database tables — the schema, Q4)

Purpose: define every database table as a Python class. This is the source of truth for the relational design.

Imports

Import	Source	What it is
<code>import enum</code>	 <code>stdlib</code>	Lets us define fixed sets of choices (roles, permission levels).
<code>import uuid</code>	 <code>stdlib</code>	Generates unique random IDs for primary keys.
<code>from datetime import datetime</code>	 <code>stdlib</code>	Timestamps (<code>created_at</code> , <code>updated_at</code>).
<code>from sqlalchemy import Column, DateTime, Enum, ForeignKey, Integer, String, Text, UniqueConstraint</code>	 <code>third-party</code>	The building blocks of a table: <code>Column</code> = a column; <code>String/Integer/Text/DateTime/Enum</code> = column types ; <code>ForeignKey</code> = a link to another table's row; <code>UniqueConstraint</code> = "this combination must be unique."
<code>from sqlalchemy.orm import relationship</code>	 <code>third-party</code>	Declares a navigable link between models (e.g. <code>document.versions</code>) without writing manual joins.
<code>from app.database import Base</code>	 <code>internal</code> (<code>app/database.py</code>)	The base class every table inherits from.

Key blocks

```
def _uuid() -> str:  
    return str(uuid.uuid4())
```

Helper that returns a fresh random UUID as a string; used as the default for every `id` column, so new rows get a unique id automatically.

```
class Role(str, enum.Enum):  
    admin = "admin"  
    editor = "editor"  
    viewer = "viewer"
```

An **enum** (fixed list of allowed values) for a user's global role. Inheriting from `str` too means the values behave like the strings `"admin"` / `"editor"` / `"viewer"` (easy to store and serialize). `PermissionLevel` (owner/editor/viewer) is the same idea but for per-document access.

```
class User(Base):  
    __tablename__ = "users"  
    id = Column(String, primary_key=True, default=_uuid)  
    email = Column(String, unique=True, nullable=False, index=True)  
    ...  
    documents = relationship("Document", back_populates="owner")
```

A table model. `__tablename__` is the SQL table name. Each `Column(...)` is a field: `primary_key=True` (the row's unique id), `unique=True` (no duplicate emails), `nullable=False` (required), `index=True` (build an index for fast lookups). `relationship(...)` is **not a column** — it's a convenient Python link: `user.documents` gives all that user's documents; `back_populates="owner"` ties it to `Document.owner` so both sides stay in sync.

The seven tables and what each is for: - `User` — accounts + global role. - `Space` — a workspace grouping documents. - `Document` — the current state of an article (`current_version` points at the latest version; `tags` is a comma string for quick filtering). - `DocumentVersion` — an **immutable snapshot** per save; `UniqueConstraint("document_id","version")` guarantees one row per (doc, version) = clean history (Q3 versioning). - `Permission` — the per-document ACL; `UniqueConstraint("document_id","user_id")` = one access row per user per doc. - `Comment` — collaboration messages on a document. - `CollaborationLog` — append-only audit trail (`action` + `detail` + timestamp), indexed by time for fast activity feeds.

`cascade="all, delete-orphan"` on the relationships means deleting a document also deletes its versions/permissions/comments (no orphans left behind).

File 4 — `app/schemas.py` (API request/response shapes)

Purpose: define, with Pydantic, the exact JSON that goes **into** and **out of** the API — and validate it automatically. (Models = database shape; schemas = API shape. They're deliberately separate.)

Imports

Import	Source	What it is
<code>from datetime import datetime</code>	 <code>stdlib</code>	Type for timestamp fields in responses.
<code>from typing import List, Optional</code>	 <code>stdlib</code>	<code>List[X]</code> = a list of X; <code>Optional[X]</code> = X or <code>None</code> .
<code>from pydantic import BaseModel, EmailStr, Field</code>	 third-party (<code>pydantic</code>)	<code>BaseModel</code> = base class for a validated data shape; <code>EmailStr</code> = a string that must be a valid email; <code>Field</code> = add rules/metadata (e.g. min length).
<code>from app.models import PermissionLevel, Role</code>	 internal (<code>app/models.py</code>)	Reuse the same enums so the API accepts exactly the DB's allowed values.

Key blocks

```
class UserCreate(BaseModel):
    email: EmailStr
    full_name: str
    password: str = Field(min_length=6)
    role: Role = Role.editor
```

The shape of the **register** request body. If a client sends a bad email or a 3-character password, FastAPI/Pydantic auto-reject it with a 422 error **before** our code runs. `role` defaults to `editor`.

```
class UserOut(BaseModel):
    id: str
    email: EmailStr
    ...
    class Config:
        from_attributes = True
```

The shape of a **user in responses** — note it has **no password field**, so we never leak it. `from_attributes = True` lets FastAPI build this directly from a SQLAlchemy `User` object (reading attributes), not just from a dict.

The rest follow the same pattern: `DocumentCreate/Update/Out` , `VersionOut` , `PermissionGrant` , `CommentCreate/Out` , and the search shapes `SearchHit` / `SearchResponse` (query, total, took_ms, cached, hits). Each is just "what fields, what types, in or out."

File 5 — `app/services/inverted_index.py` (the search algorithm, Q5)

Purpose: a from-scratch keyword search engine (TF-IDF + cosine similarity) using only the standard library. It's the answer to Q5 and the offline fallback when Elasticsearch is down.

Imports

Import	Source	What it is
<code>from __future__ import annotations</code>	 <code>stdlib</code>	Lets us write type hints that reference types defined later; harmless compatibility helper.
<code>import math</code>	 <code>stdlib</code>	<code>log</code> and <code>sqrt</code> for the IDF weight and vector lengths.
<code>import re</code>	 <code>stdlib</code>	Regular expressions — to split text into words.
<code>from collections import defaultdict</code>	 <code>stdlib</code>	A dict that auto-creates a default value for missing keys (so we can do <code>postings[term][doc] += 1</code> without checking existence).
<code>from typing import Dict, List, Tuple</code>	 <code>stdlib</code>	Type hints for readability.

Key blocks

```
def tokenize(text: str) -> List[str]:
    return [t for t in _TOKEN_RE.findall(text.lower()) if t not in STOP_WORDS]
```

Analysis step: lowercase the text, pull out word-like tokens with a regex, and drop common stop-words (`the` , `is` , ...). The *same* function is applied to documents and to queries, so terms line up.

```
class InvertedIndex:
    def __init__(self):
        self.postings = defaultdict(dict) # term -> {doc_id: count}
        self.doc_length = {}             # doc_id -> number of tokens
        self.titles = {}                  # doc_id -> title
        self.raw = {}                     # doc_id -> original text
```

The index's data: the **postings map** (`term` → `which docs contain it and how often`) plus bookkeeping for scoring and showing results.

```
def add(self, doc_id, text, title=""):
    if doc_id in self.doc_length: self.remove(doc_id) # re-index cleanly
    tokens = tokenize(text)
    self.doc_length[doc_id] = len(tokens) or 1
    ...
    for term, freq in term_freq.items():
        self.postings[term][doc_id] = freq
```

Indexing: tokenize, record how many times each term appears in this document, and store that in the postings map.

```
def _idf(self, term):
    df = len(self.postings.get(term, {}))
    return math.log(1 + self.num_docs / df) if df else 0.0
```

IDF (inverse document frequency): rare terms (in few documents) get a higher weight; ubiquitous terms get a low weight.

```
def search(self, query, limit=10):
    q_tokens = tokenize(query)
    q_vec = {t: tf * self._idf(t) for t, tf in q_tf.items()} # query as TF-IDF vector
    ...
    for term, q_weight in q_vec.items():
        for doc_id, tf in self.postings.get(term, {}).items():
            d_weight = (tf / self.doc_length[doc_id]) * idf
            scores[doc_id] += q_weight * d_weight # dot product
    ...
    cosine = dot / (q_norm * d_norm) # normalise by lengths
```

Ranking: turn the query into a TF-IDF vector, only touch documents that actually contain a query term (that's the speed win), compute the dot product, then divide by the vector lengths to get **cosine similarity** (so long documents aren't unfairly favoured). Results are sorted by score. `_snippet(...)` builds the little preview around the first matched word.

This is the same family of maths as Lucene/Elasticsearch's BM25, written out explicitly.

File 6 — `app/cache.py` (Redis cache + in-memory fallback)

Purpose: store/fetch cached values in Redis; if Redis is missing, transparently use an in-process dictionary instead.

Imports

Import	Source	What it is
<code>import json</code>	 stdlib	Convert Python values ↔ JSON strings (Redis stores strings).
<code>import logging</code>	 stdlib	Print informational/warning messages.
<code>from typing import Any, Optional</code>	 stdlib	<code>Any</code> = any type; <code>Optional</code> = value or <code>None</code> .
<code>from app.config import settings</code>	 internal	For <code>redis_url</code> and <code>cache_ttl_seconds</code> .
<code>import redis</code> (inside <code>_build_client</code>)	 third-party (<code>redis</code>)	The Redis client library — imported lazily inside the function so the app still works if the package isn't installed.

Key blocks

```
class _MemoryCache:
    def get(...); def set(...); def delete(...); def scan_iter(...)
```

A tiny **fallback** that mimics the few Redis methods we use, backed by a plain dict. The leading underscore signals "internal/private."

```
def _build_client():
    try:
        import redis
        client = redis.Redis.from_url(settings.redis_url, decode_responses=True)
        client.ping()
        return client
    except Exception:
        return _MemoryCache()
_client = _build_client()
```

Try to connect to Redis (and `ping()` to confirm it's alive). If anything fails, return the in-memory cache instead. `_client` is decided **once at startup** — the rest of the app doesn't know or care which backend it got (that's why `/health` reports the chosen one).

```
def cache_get(key): ... json.loads(...)
def cache_set(key, value, ttl=None): ... json.dumps(...) ex=ttl or settings.cache_ttl_seconds
def cache_delete(*keys): ...
def invalidate_prefix(prefix): ... # delete every key starting with prefix, e.g. "search:"
```

The public helpers. Values are JSON-encoded on the way in and decoded on the way out. `ex=` sets the **TTL** (auto-expiry). `invalidate_prefix("search:")` wipes all cached search results at once after a write — this is how we avoid serving stale results.

File 7 — `app/search.py` (Elasticsearch layer + fallback)

Purpose: own the search index — create it, add/update/delete documents, and run queries. Falls back to the Python `InvertedIndex` if Elasticsearch is unavailable.

Imports

Import	Source	What it is
<code>import logging</code>	 <code>stdlib</code>	Log connection status / errors.
<code>from typing import List, Optional</code>	 <code>stdlib</code>	Type hints.
<code>from app.config import settings</code>	 <code>internal</code>	ES URL + index name.
<code>from app.services.inverted_index import InvertedIndex</code>	 <code>internal</code>	The fallback/offline engine.
<code>from elasticsearch import Elasticsearch</code> (inside <code>_connect</code>)	 <code>third-party</code> (<code>elasticsearch</code>)	The official ES client — imported lazily so the app runs without it.

Key blocks

```
_INDEX_MAPPING = { "mappings": { "properties": {
    "title": {"type":"text","analyzer":"english", ...},
    "content": {"type":"text","analyzer":"english"},
    "tags": {"type":"keyword"}, ... }}}
```

The **schema for the search index** — tells Elasticsearch to treat `title` / `content` as English full-text (stemming, etc.) and `tags` /ids as exact-match keywords.

```
class SearchEngine:
    def __init__(self):
        self.es = self._connect()          # real ES client or None
        self._fallback = InvertedIndex()    # always present
        if self.es is not None: self._ensure_index()
```

On startup, try to connect to ES; always keep a local index too. `_ensure_index()` creates the ES index (with the mapping) if it doesn't exist yet.

```
def index_document(self, doc):
    self._fallback.add(doc["id"], f"{doc['title']} {doc['content']}", title=doc["title"])
    if self.es is None: return
    self.es.index(index=self.index_name, id=doc["id"], document={...})
```

Indexing a document writes to *both* the local index and (if present) ES, keyed by the document id (so re-indexing overwrites cleanly — idempotent).

```
def _search_es(self, query, limit, allowed_ids):
    must = [{"multi_match": {"query": query, "fields": ["title^3", "content"], "fuzziness": "AUTO"}}]
    bool_query = {"bool": {"must": must}}
    if allowed_ids is not None:
        bool_query["bool"]["filter"] = [{"ids": {"values": list(allowed_ids)}}]
    resp = self.es.search(index=..., query=bool_query, size=limit, highlight={...})
```

The query: match the words in `title` (weighted x3) and `content`, allow small typos (`fuzziness`), and — crucially — **filter to only the document ids the user may see** (`allowed_ids`). That's permission enforcement *inside* the query. The comment in the code notes we pass `query=` / `highlight=` / `size=` as explicit arguments because the old `body=` style silently broke in the 8.x client. The results are mapped to a simple list of `{document_id, title, score, highlight}`. `_search_fallback(...)` does the same against the local index when ES is absent.

File 8 — `app/auth.py` (security: passwords, JWT, access control)

Purpose: hash/verify passwords, create/check JWT tokens, identify the current user, and enforce permissions.

Imports

Import	Source	What it is
<code>from datetime import datetime, timedelta</code>	 stdlib	Compute the token's expiry time.
<code>from typing import Optional</code>	 stdlib	"value or None".
<code>import bcrypt</code>	 third-party (<code>bcrypt</code>)	Securely hash and check passwords.
<code>from fastapi import Depends, HTTPException, status</code>	 third-party (<code>fastapi</code>)	<code>Depends</code> = dependency injection; <code>HTTPException</code> = return an HTTP error; <code>status</code> = named status codes (401, 403).
<code>from fastapi.security import OAuth2PasswordBearer</code>	 third-party	Reads the <code>Authorization: Bearer <token></code> header and powers the Swagger "Authorize" button.
<code>from jose import JWTError, jwt</code>	 third-party (<code>python-jose</code>)	Encode/decode JWT tokens; <code>JWTError</code> = raised on an invalid token.
<code>from sqlalchemy.orm import Session</code>	 third-party	DB session type.
<code>from app.config import settings</code>	 internal	Secret key, algorithm, token lifetime.
<code>from app.database import get_db</code>	 internal	DB-session dependency.
<code>from app.models import Document, Permission, PermissionLevel, Role, User</code>	 internal	The tables/enums we check against.

Key blocks

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")
```

Declares "tokens are obtained from `/auth/login`." Used as a dependency to pull the bearer token off requests.

```
def hash_password(password):  
    pw = password.encode("utf-8")[:72]  
    return bcrypt.hashpw(pw, bcrypt.gensalt()).decode("utf-8")  
def verify_password(plain, hashed): ... bcrypt.checkpw(...)
```

Hash with a random salt (`gensalt`) and check later. bcrypt only uses the first 72 bytes, so we truncate defensively. We store the **hash**, never the password.

```
def create_access_token(subject):  
    expire = datetime.utcnow() + timedelta(minutes=settings.access_token_expire_minutes)  
    payload = {"sub": subject, "exp": expire}  
    return jwt.encode(payload, settings.secret_key, algorithm=settings.algorithm)
```

Build a **JWT**: `sub` = who it's for (the user id), `exp` = when it expires. It's signed with our secret key, so it can't be forged or tampered with.

```
def get_current_user(token=Depends(oauth2_scheme), db=Depends(get_db)) -> User:
    payload = jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])
    user_id = payload.get("sub")
    ...
    user = db.query(User).filter(User.id == user_id).first()
    if user is None: raise credentials_exc
    return user
```

The **gatekeeper dependency**. Any endpoint that writes `user: User = Depends(get_current_user)` automatically: gets the token, verifies the signature/expiry (else 401), looks the user up, and hands the `User` object to the endpoint. Invalid/expired token → 401 before the endpoint runs.

```
_LEVELS = {viewer:1, editor:2, owner:3}
def effective_level(db, user, document):
    if user.role == Role.admin or document.owner_id == user.id: return owner
    perm = db.query(Permission).filter(...).first()
    return perm.level if perm else None
def require_access(db, user, document, minimum):
    level = effective_level(...)
    if level is None or _LEVELS[level] < _LEVELS[minimum]: raise 403
```

Authorization. `effective_level` works out a user's power over a document (admins and owners get full `owner` ; otherwise look up the ACL row). `require_access` converts levels to numbers and raises **403 Forbidden** unless the user meets the minimum required (viewer < editor < owner). Every protected document endpoint calls this, and search reuses the same idea.

File 9 — `app/services/document_service.py` (cross-store orchestration)

Purpose: the rules that touch several stores at once — versioning, keeping the search index in sync, clearing caches, and writing the audit log — kept in one place so the route handlers stay thin.

Imports

Import	Source	What it is
<code>from typing import List, Optional</code>	 stdlib	Type hints.
<code>from sqlalchemy.orm import Session</code>	 third-party	DB session type.
<code>from app.cache import cache_delete, invalidate_prefix</code>	 internal	Clear stale cache entries after writes.
<code>from app.models import CollaborationLog, Document, DocumentVersion, User</code>	 internal	The tables we create/update.
<code>from app.search import search_engine</code>	 internal	The shared search engine, to (re)index documents.

Key blocks

```
def create_document(db, owner, title, content, space_id, tags):
    doc = Document(...); db.add(doc); db.flush() # get doc.id
    db.add(DocumentVersion(document_id=doc.id, version=1, ...)) # first version
    log_action(db, owner.id, doc.id, "created", ...) # audit
    db.commit()
    _sync_index(doc) # add to search
    invalidate_prefix("search:") # drop stale searches
    return doc
```

Creating a document is **one transaction**: insert the document + version 1 + an audit log row, commit, then update the search index and clear search caches. `db.flush()` assigns the new id before we reference it in the version row.

```
def update_document(db, editor, doc, title, content, tags):
    if changed:
        doc.current_version += 1
        db.add(DocumentVersion(... version=doc.current_version ...)) # new immutable version
        log_action(db, editor.id, doc.id, "updated", f"v{doc.current_version}")
    db.commit(); _sync_index(doc); cache_delete(f"doc:{doc.id}"); invalidate_prefix("search:")
```

Editing **never overwrites history** — it bumps `current_version` and appends a new `DocumentVersion`, then re-indexes and clears the affected caches (this doc's read cache + all search caches).

`delete_document` removes the row (cascades delete versions/comments/permissions), removes it from the index, and clears caches. `to_dict(...)` converts a `Document` to a plain dict (tags as a list) for responses/caching.

Files 10–13 — `app/routers/*` (the HTTP endpoints)

All routers share the same shape, so here are the **shared idioms** (explained once), then each file's specifics.

Shared imports across routers: | Import | Source | What it is | |---|---|---| | `from fastapi import APIRouter,`
`Depends, HTTPException, (Query)` |  third-party | `APIRouter` = a group of endpoints; `Depends` = inject DB/user;
`HTTPException` = error responses; `Query` = declare/validate query-string params. | | `from sqlalchemy.orm import`
`Session` |  third-party | DB session type. | | `from app import auth` |  internal | `auth.get_current_user`,
`auth.require_access` | | `from app.database import get_db` |  internal | DB session dependency. | | `from`
`app.models import ...` |  internal | Tables/enums. | | `from app.schemas import ...` |  internal |
Request/response shapes. | | `from app.services import document_service as svc` |  internal | The
orchestration helpers. |

Shared idiom — every endpoint looks like:

```
@router.post("/documents", response_model=DocumentOut, status_code=201)
def create(payload: DocumentCreate, db: Session = Depends(get_db), user: User = Depends(auth.get_
...)
```

- `@router.post(...)` — the HTTP method + path + (optional) response shape + status code.
- `payload: DocumentCreate` — FastAPI reads & validates the JSON body into that schema.
- `db = Depends(get_db)` — injects a DB session.
- `user = Depends(auth.get_current_user)` — **requires a valid token** and injects the user.

File 10 — `app/routers/auth.py`

- `from fastapi.security import OAuth2PasswordRequestForm` (■) — reads the form-encoded `username / password` for login.
- `POST /auth/register` — reject if the email exists, hash the password (`auth.hash_password`), save the `User`, return `UserOut`.
- `POST /auth/login` — look up by email, `auth.verify_password`; on success return a JWT via `auth.create_access_token`; on failure 401. (Login uses a **form**, not JSON, because that's the OAuth2 standard.)
- `GET /auth/me` — just returns the injected `current` user (proves the token works).

File 11 — `app/routers/documents.py`

- Imports also include `from app.cache import cache_get, cache_set, invalidate_prefix` (■) for read-caching and cache busting.
- `_get_doc_or_404(...)` — helper: fetch a document or raise 404.
- `POST /documents` — `svc.create_document(...)` (writes doc + v1 + index + log).
- `GET /documents/{id}` — check the Redis read-cache first; on a miss load it, **enforce** `require_access(..., viewer)`, cache it. (Even on a cache hit it re-checks permission against the DB, so cached data can't leak.)
- `PUT /documents/{id}` — require **editor**, then `svc.update_document(...)` (new version).
- `DELETE /documents/{id}` — require **owner**, then `svc.delete_document(...)`.
- `GET /documents/{id}/versions` — require viewer, return the revision history newest-first.
- `POST /documents/{id}/share` — require **owner**, create/update a `Permission` row, log it, and `invalidate_prefix("search:")` so the newly-granted user immediately sees the doc in search.

File 12 — `app/routers/search.py`

- Extra imports: `import hashlib`, `import time` (■) for the cache key and latency timing; `from fastapi import Query` (■); `from app.cache import cache_get, cache_set` (■).
- `_visible_document_ids(db, user)` — returns the set of doc ids the user may see (all, if admin; otherwise owned ∪ shared). This set becomes the ES `ids` filter.
- `GET /search` — build a **per-user cache key** (`"search:" + sha256(user_id+limit+query)`); return the cached answer if present; otherwise compute visible ids, call `search_engine.search(q, limit, allowed_ids=...)`, time it (`took_ms`), cache it, and return. The per-user key is why one user's results never leak to another.

File 13 — `app/routers/collaboration.py`

- `POST /documents/{id}/comments` — require viewer, save a `Comment`, log a **"commented"** action.
- `GET /documents/{id}/comments` — require viewer, return comments oldest-first.
- `GET /documents/{id}/activity` — require viewer, return the latest `CollaborationLog` rows (the activity/audit feed) newest-first.

File 14 — `app/main.py` (the entry point)

Purpose: assemble the FastAPI app — register all routers, mount the web UI, create tables, expose `/health`.

Imports

Import	Source	What it is
<code>import logging</code>	 stdlib	Configure log output.
<code>import pathlib</code>	 stdlib	Build the filesystem path to the <code>static/</code> folder.
<code>from fastapi import FastAPI</code>	 third-party	The application class.
<code>from fastapi.responses import FileResponse</code>	 third-party	Return a file (the UI's <code>index.html</code>) from the root URL.
<code>from fastapi.staticfiles import StaticFiles</code>	 third-party	Serve a whole folder of static files (the web app).
<code>from app import __version__</code>	 internal (<code>app/__init__.py</code>)	Version string shown in the docs.
<code>from app.cache import _client</code>	 internal	To report the active cache backend in <code>/health</code> .
<code>from app.database import Base, engine</code>	 internal	To create the tables.
<code>from app.routers import auth/collaboration/documents/search</code>	 internal	The four endpoint groups.
<code>from app.search import search_engine</code>	 internal	To report the active search backend in <code>/health</code> .
<code>from app import models</code>	 internal	Imported for its side effect : it registers all tables on <code>Base</code> so <code>create_all</code> can build them.

Line by line

```
Base.metadata.create_all(bind=engine)
```

Create any missing tables in the database at startup (fine for a project; production would use migrations).

```
app = FastAPI(title="KnowledgeHub", version=__version__, description=...)
```

Create the application object (this also powers the auto-generated `/docs`).

```
app.include_router(auth_router.router)
app.include_router(documents_router.router)
app.include_router(collab_router.router)
app.include_router(search_router.router)
```

Plug each router's endpoints into the app.

```
_STATIC_DIR = pathlib.Path(__file__).parent / "static"
app.mount("/app", StaticFiles(directory=_STATIC_DIR, html=True), name="webapp")
```

Serve the **web UI** folder at `/app` (`html=True` serves `index.html` automatically). Same origin as the API, so the browser needs no CORS exceptions.

```
@app.get("/")           -> returns index.html (redirect-to-UI)
@app.get("/api")         -> small JSON describing the service
@app.get("/health")      -> {"status","search_backend","cache_backend"}
```

`/health` reports which backends are actually live by reading `search_engine.backend` and the class name of the cache `_client` — that's how you confirm whether you're on the full stack (`elasticsearch` / `Redis`) or the fallbacks.

File 15 — `app/static/index.html` (the web UI)

Not Python — a single HTML file with embedded CSS and vanilla JavaScript. It has **no imports/libraries** (zero build step). Its JavaScript uses the browser's built-in `fetch()` to call the same API endpoints described above, sending the JWT in the `Authorization` header. Key functions: `login()` / `register()` (get + store the token), `createDoc()`, `doSearch()` (renders ranked hits with highlights), `openDoc()` (loads content + versions + comments), and `addComment()`. State is just two variables (`TOKEN`, `CURRENT_DOC`); the token is kept in `localStorage` so a refresh keeps you logged in.

One-screen mental model

```
config.py      -> settings (env-driven)
database.py    -> engine + session + Base
models.py      -> tables (User, Document, Version, Permission, Comment, Log)
schemas.py     -> API request/response shapes (Pydantic)
inverted_index -> the from-scratch TF-IDF search (Q5) + offline fallback
cache.py       -> Redis (or in-memory) caching
search.py      -> Elasticsearch (or fallback) indexing + querying
auth.py        -> bcrypt + JWT + permission checks
document_service -> versioning + index sync + cache busting + audit
routers/*      -> the HTTP endpoints (auth, documents, search, collaboration)
main.py        -> wires it all together + serves the web UI + /health
static/index.html -> the browser app
```

Every arrow of dependency points "downward": routers use services/auth/cache/search, which use models/config/database. Nothing low-level imports the routers — a clean, layered design.